

# Appendix

## A Incorporating Guidance During Sampling

In an unconditional diffusion model, test-time guidance [7, 12] constrains trajectories to specific environments and start/goal configurations. While prior works [2, 6] rely on test-time guidance for collision avoidance and endpoint constraints, we use only conditional diffusion models and trajectory optimization for strict constraint satisfaction. Here, we also include an ablation study on the complementary use of guidance steps during sampling to enhance motion planning performance.

**Guidance Function Implementation** Algorithm 3 describes our procedure for computing cost gradients. As in MPD [2], the guidance function includes collision and smoothness costs as described in Section IV-B. Collision costs are computed using cuRobo [10], while smoothness costs employ a Gaussian Process prior [1, 2]. To ensure stability during guidance, we smooth the sampled trajectory with a Gaussian kernel ( $\sigma = 4.0$ ) before computing costs, allowing collision-cost gradients to affect neighboring points (line 1). We then compute the clamped collision cost ( $d_{\max} = 0.1$ ) and the smoothness cost, summing them as  $k_{\text{smooth}}c_{\text{smooth}} + k_{\text{coll}}c_{\text{coll}}$ , with  $k_{\text{smooth}} = 1e - 9$  and  $k_{\text{coll}} = 1e - 2$  (line 2-4). Gradients are computed with PyTorch [9], clamped ( $g_{\max} = 1.0$ ) to prevent erratic updates, zeroed at endpoints, and directly added to the trajectory (line 5-6).

**Results** Figure 6 compares our model with a variant that includes guidance steps during denoising iterations. Overall, guided-sampling enhances the quality of initial trajectories (black dots represent PRESTO without guidance, and gray dots represent PRESTO with guidance). For example, the success rate of the denoised trajectory before optimization reaches 92.8% in Level 4, compared to 51.1% for PRESTO without guidance. However, incorporating guidance requires evaluating cost gradients at each diffusion iteration, resulting in computational overhead. In Level 3, this overhead accumulates to an average of 0.38 seconds, indicating that the additional cost of guidance steps may occasionally degrade performance within a given time frame. Despite this, guidance generally improves performance across Levels 1-4 in all three metrics: success rate, collision rate, and penetration depth, given the same number of trajectory optimization iterations.

We also report the effects of guidance on variants of our model in Figure 5. Consistent with the results in Figure 6, performance across all baselines improves when guidance steps are applied compared to the original results in Figure 5. Notably, the success rate gap between PRESTO and its variants widens with the application of guidance. For instance, across Level 1-4, the gap between PRESTO and the closest baseline (*Point-Cloud Conditioning*) increases from 2.4% to 4.0%. As PRESTO generates higher-quality initial trajectories with smaller penetration depths, spurious collisions are resolved with just a few guidance steps, leading to greater success rate gains compared to the ablations of PRESTO.

---

**Algorithm 3** Guidance Step with Cost Gradients.

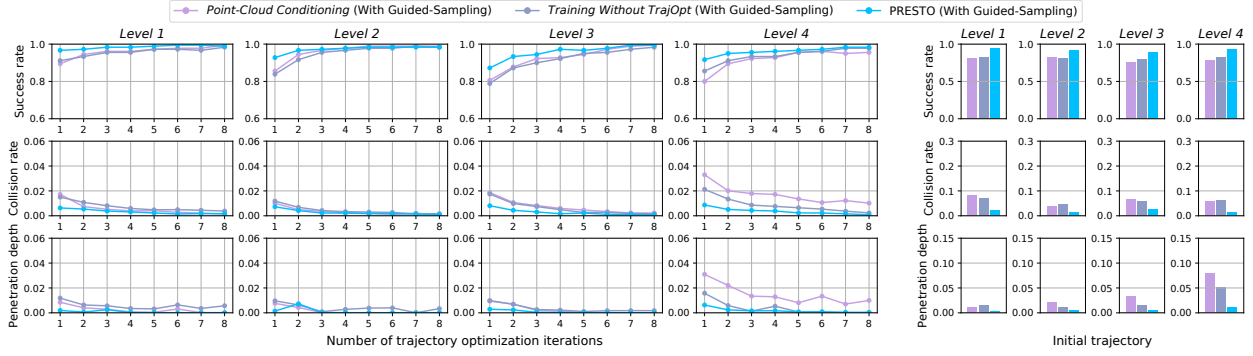
---

**Require:** Trajectory  $\tau$ , Smoothing Kernel  $\mathcal{K}$ , Environment  $\mathcal{G}$

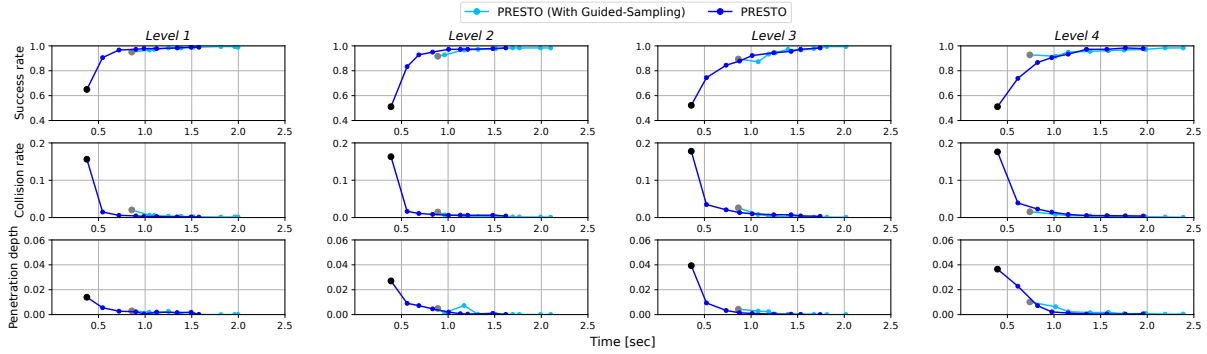
**Ensure:** Output trajectory  $\tau_{\text{guide}}$

```
1:  $\tilde{\tau} = \text{Convolve}(\text{UnNormalize}(\tau), \mathcal{K})$ 
   // Computing costs
2:  $c_{\text{coll}} = \min(\text{CollisionCost}(\tilde{\tau}, \mathcal{G}), d_{\max})$ 
3:  $c_{\text{smooth}} = \text{GPCost}(\tau)$ 
4:  $c_{\text{guide}} = k_{\text{smooth}}c_{\text{smooth}} + k_{\text{coll}}c_{\text{coll}}$ 
   // Applying guidance gradients
5:  $g_{\text{guide}} = \text{Clamp}(\nabla_{\tau}(c_{\text{guide}}), g_{\max})$ 
6:  $\tau_{\text{guide}} = \tau - g_{\text{guide}}$ 
7: return  $\tau_{\text{guide}}$ 
```

---



**Figure 5: Ablation studies with guided-sampling.** We report the success rate (%), collision rate (%), and penetration depth (m) averaged across 180 problems for PRESTO and the self-variant baselines with guided-sampling. **(Left)** We show performance changes with varying post-processing iterations. **(Right)** We present performance of trajectories directly generated by the diffusion model, without post-processing.



**Figure 6: Results with guided-sampling.** We report the success rate (%), collision rate (%), and penetration depth (m) averaged across 180 problems for PRESTO and the self-variant baselines. Black dots represent PRESTO without post-processing, while gray dots represent PRESTO with Guided Sampling, also without post-processing.

## B Computation Time Details

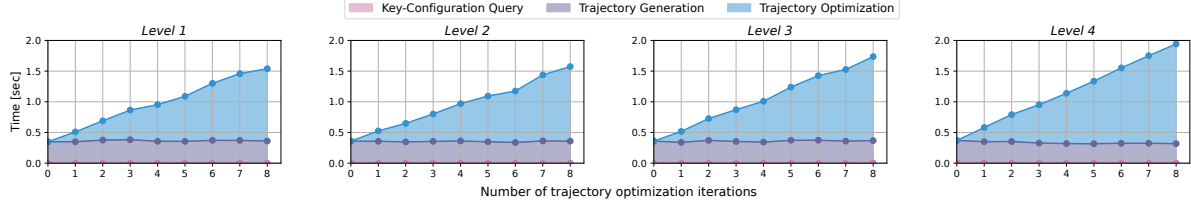
We provide computation time details for PRESTO in Section V and present them in Figure 7. The key-configuration query incurs an average overhead of 0.658 milliseconds, which is negligible compared to trajectory generation in the diffusion model, averaging 0.345 seconds, and trajectory optimization during post-processing, which scales linearly with the number of iterations. The performance improvements of PRESTO, leveraging an environment representation based on key configurations, over the *Point-Cloud Conditioning* baseline in Section V-C demonstrate its effectiveness in enhancing representation quality without compromising computational speed. This makes it particularly well-suited for scenarios with limited computational resources.

## C Point-Cloud Encoder Architecture

For our ablation with point-cloud inputs in Section V-C, we design the point-cloud encoder based on recent patch-based transformers [8, 11]. We divide the  $\mathbb{R}^{1024 \times 3}$  point cloud into 8 patches using farthest-point sampling and k-nearest neighbors ( $k = 128$ ). Each patch is normalized, flattened, and projected into shape embeddings via a 3-layer MLP with GeLU and Layer Normalization. Positional embeddings, computed from patch centers using a 2-layer MLP, are added before processing with a 4-layer transformer to extract geometric features, which serve as additional input tokens for the DiT in the diffusion model.

## D Background

**Denoising Diffusion Probabilistic Models** The diffusion model involves a forward diffusion process, which starts from a perfect solution (in our case a collision-free trajectory) and gradually injects noise until reaching an isotropic Gaussian distribution, and a reverse diffusion process, which learns to generate data through gradual denoising. Let



**Figure 7: Breakdown of computation time for PRESTO.** We report the average computation time (sec) across 180 problems for PRESTO and provide a breakdown of the time taken for each step: the key-configuration query for environment representation, trajectory generation using the diffusion model, and trajectory optimization during post-processing.

$i \in \{0, 1, \dots, N\}$  denote a denoising step. The data generation process using a diffusion model involves iteratively denoising noise  $\tau_i$  until reaching a denoised sample  $\tau_0$ , where  $\tau$  is a sequence of robot configurations in this work. The reverse diffusion process is given by:

$$p_{\theta}(\tau_{0:T}) = p(\tau_T) \prod_{i=1}^N p_{\theta}(\tau_{i-1}|\tau_i),$$

where  $p_{\theta}(\tau_{i-1}|\tau_i) = \mathcal{N}(\tau_{i-1}; \mu_{\theta}(\tau_i, i), \Sigma_{\theta}(\tau_i, i))$ , as proven by Ho *et al.* [4] under some mild conditions. Ho *et al.* [4] show that  $\mu_{\theta}(\tau_i, i)$  can be modeled as a function  $\epsilon_{\theta}(\tau_i, i)$ , which predicts a noise value from a sample at the  $i$ -th denoising step. In addition, they show that the training objective can be simplified  $\min_{\theta} \|\epsilon_{\theta}(\tau_i, i) - \epsilon_i\|^2$ , and the diffusion model can be trained by minimizing this term.

**Conditional Diffusion Models** We convert the unconditional form of  $p_{\theta}(\tau_{i-1}|\tau_i)$  to be conditioned on our environment representation. We adopt the methods proposed by Dhariwal and Nichol [3], whose objective is to generate an image conditioned on the class. Specifically, they demonstrate that the term of the conditional reverse diffusion process can be factorized as follows:  $p_{\theta, \phi}(\tau_{i-1}|\tau_i, \phi) \propto p_{\theta}(\tau_{i-1}|\tau_i) p_{\phi}(\phi|\tau_{i-1})$ , where  $\phi$  is the conditioning variable, which in our case will be an environment representation. Under some mild conditions, they show that after applying the logarithm, we get  $\log(p_{\theta}(\tau_{i-1}|\tau_i) p_{\phi}(\phi|\tau_{i-1})) \approx \log p(u) + \text{const}$ , where  $u \sim \mathcal{N}(\tau_{i-1}; \mu_{\theta}(\tau_i, i) + g\Sigma_{\theta}(\tau_i, i), \Sigma_{\theta}(\tau_i, i))$  and  $g = \nabla_{\tau_{i-1}} \log p_{\phi}(\phi|\tau_{i-1})|_{\tau_{i-1}=\mu_{\theta}(\tau_i, i)}$ . Notice that the mean parameter of the Gaussian term in  $u$  includes an additional component,  $g\Sigma_{\theta}(\tau_i, i)$ , which is not a part of the mean parameter of  $p_{\theta}(\tau_{i-1}|\tau_i)$ . This mathematical derivation implies that the diffusion model can generate a sample conditioned on  $\phi$  by the gradients  $g$ . Based on this, we particularly use a classifier-free guidance diffusion model to implement our motion generation. It is known that conditional diffusion steps can be run by incorporating the probabilities from both a conditional and an unconditional diffusion model [5], as  $\nabla \log p_{\phi}(\phi|\tau) = \nabla \log p_{\phi}(\tau|\phi) - \nabla \log p_{\phi}(\tau)$ . Therefore, by plugging into the conditional probabilities, conditional guidance can be trained together with the diffusion models, eliminating the need for a separate classifier.

## References

- [1] T. D. Barfoot, C. H. Tong, and S. Särkkä, “Batch continuous-time trajectory estimation as exactly sparse gaussian process regression,” in *Robotics: Science and Systems*, 2014.
- [2] J. Carvalho, A. T. Le, M. Baierl, D. Koert, and J. Peters, “Motion planning diffusion: Learning and planning of robot motions with diffusion models,” in *IEEE/RSJ International Conference on Intelligent Robots and Systems*, 2023.
- [3] P. Dhariwal and A. Nichol, “Diffusion models beat gans on image synthesis,” in *Advances in Neural Information Processing Systems*, 2021.
- [4] J. Ho, A. Jain, and P. Abbeel, “Denoising diffusion probabilistic models,” in *Advances in Neural Information Processing Systems*, 2020.
- [5] J. Ho and T. Salimans, “Classifier-free diffusion guidance,” *arXiv preprint arXiv:2207.12598*, 2022.
- [6] S. Huang *et al.*, “Diffusion-based generation, optimization, and planning in 3d scenes,” in *IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2023.
- [7] M. Janner, Y. Du, J. Tenenbaum, and S. Levine, “Planning with diffusion for flexible behavior synthesis,” in *International Conference on Machine Learning*, 2022.

- [8] Y. Pang, W. Wang, F. E. Tay, W. Liu, Y. Tian, and L. Yuan, “Masked autoencoders for point cloud self-supervised learning,” in *European Conference on Computer Vision*, 2022.
- [9] A. Paszke *et al.*, “PyTorch: An imperative style, high-performance deep learning library,” in *Advances in Neural Information Processing Systems*, 2019.
- [10] B. Sundaralingam *et al.*, “cuRobo: Parallelized collision-free minimum-jerk robot motion generation,” *arXiv preprint arXiv:2310.17274*, 2023.
- [11] X. Yu, L. Tang, Y. Rao, T. Huang, J. Zhou, and J. Lu, “Point-bert: Pre-training 3d point cloud transformers with masked point modeling,” in *IEEE Conference on Computer Vision and Pattern Recognition*, 2022.
- [12] M. Zhang *et al.*, “Motiondiffuse: Text-driven human motion generation with diffusion model,” *arXiv preprint arXiv:2208.15001*, 2022.